

COLLEGE EVENT MANAGEMENT SYSTEM

19 Queries / Steps — MySQL Source Code

1. Create Student Table

```
CREATE TABLE Student
(
    Student_ID INT PRIMARY KEY,
    Student_Name VARCHAR(50) NOT NULL,
    Branch VARCHAR(30),
    `Year` INT,
    Email VARCHAR(100) UNIQUE
);
```

2. Create Organizer Table

```
CREATE TABLE Organizer
(
    Organizer_ID INT PRIMARY KEY,
    Organizer_Name VARCHAR(50) NOT NULL,
    Department VARCHAR(50),
    Contact VARCHAR(15)
);
```

3. Create Event Table

```
CREATE TABLE `Event`
(
    Event_ID INT PRIMARY KEY,
    Event_Name VARCHAR(100) NOT NULL,
    Event_Date DATE,
    Venue VARCHAR(100),
    Organizer_ID INT,
    Max_Seats INT,
    FOREIGN KEY (Organizer_ID)
    REFERENCES Organizer(Organizer_ID)
);
```

4. Create Registration Table

```
CREATE TABLE Registration
(
    Registration_ID INT PRIMARY KEY,
    Student_ID INT,
    Event_ID INT,
    Registration_Date DATE,
    FOREIGN KEY (Student_ID)
    REFERENCES Student(Student_ID),
    FOREIGN KEY (Event_ID)
    REFERENCES `Event`(Event_ID),
    UNIQUE(Student_ID, Event_ID)
);
```

5. Create Attendance Table

```
CREATE TABLE Attendance
(
    Attendance_ID INT PRIMARY KEY,
    Registration_ID INT UNIQUE,
    Status VARCHAR(20),
    FOREIGN KEY (Registration_ID)
    REFERENCES Registration(Registration_ID)
);
```

6. Insert Student Data

```
INSERT INTO Student
(Student_ID, Student_Name, Branch, `Year`, Email)
VALUES
(101, 'Amit Sharma', 'IT', 2, 'amit@gmail.com'),
(102, 'Rahul Patil', 'CSE', 2, 'rahul@gmail.com'),
(103, 'Priya Singh', 'IT', 3, 'priya@gmail.com'),
(104, 'Sneha Joshi', 'ENTC', 2, 'sneha@gmail.com'),
(105, 'Rohan Verma', 'CSE', 3, 'rohan@gmail.com');
```

7. Insert Organizer Data

```

INSERT INTO Organizer
(Organizer_ID, Organizer_Name, Department, Contact)
VALUES
(1, 'Tech Club', 'Information Technology', '9876543210'),
(2, 'Cultural Club', 'Arts', '9876543211'),
(3, 'Sports Club', 'Physical Education', '9876543212');

```

8. Insert Event Data

```

INSERT INTO `Event`
(Event_ID, Event_Name, Event_Date, Venue, Organizer_ID, Max_Seats)
VALUES
(201, 'Coding Competition', '2026-10-10', 'Computer Lab', 1, 50),
(202, 'Dance Competition', '2026-10-15', 'Auditorium', 2, 100),
(203, 'Cricket Tournament', '2026-10-20', 'College Ground', 3, 80),
(204, 'Quiz Competition', '2026-10-25', 'Seminar Hall', 1, 60);

```

9. Insert Registration Data

```

INSERT INTO Registration
(Registration_ID, Student_ID, Event_ID, Registration_Date)
VALUES
(301, 101, 201, '2026-09-20'),
(302, 102, 201, '2026-09-21'),
(303, 103, 202, '2026-09-22'),
(304, 104, 203, '2026-09-23'),
(305, 105, 204, '2026-09-24');

```

10. Insert Attendance Data

```

INSERT INTO Attendance
(Attendance_ID, Registration_ID, Status)
VALUES
(401, 301, 'Present'),
(402, 302, 'Present'),
(403, 303, 'Absent'),
(404, 304, 'Present'),
(405, 305, 'Present');

```

11. Basic SQL Queries

```

SELECT * FROM Student;
SELECT * FROM `Event`;
SELECT * FROM Student WHERE Branch = 'IT';
SELECT * FROM `Event` WHERE Venue = 'Auditorium';

```

```

UPDATE Student
SET Email = 'amit.sharma@gmail.com'
WHERE Student_ID = 101;

```

```

SELECT * FROM Student WHERE Student_ID = 101;

```

12. JOIN Queries

```

SELECT Student.Student_Name, `Event`.Event_Name
FROM Registration
INNER JOIN Student
    ON Registration.Student_ID = Student.Student_ID
INNER JOIN `Event`
    ON Registration.Event_ID = `Event`.Event_ID;

```

```

SELECT `Event`.Event_Name, Organizer.Organizer_Name,
    Organizer.Department
FROM `Event`
INNER JOIN Organizer
    ON `Event`.Organizer_ID = Organizer.Organizer_ID;

```

```

SELECT Registration.Registration_ID, Student.Student_Name,
    `Event`.Event_Name, Registration.Registration_Date
FROM Registration
INNER JOIN Student
    ON Registration.Student_ID = Student.Student_ID
INNER JOIN `Event`
    ON Registration.Event_ID = `Event`.Event_ID;

```

13. Aggregate Functions

```

SELECT COUNT(*) AS Total_Students FROM Student;
SELECT COUNT(*) AS Total_Events FROM `Event`;

```

14. GROUP BY and HAVING

```
SELECT `Event`.Event_Name,  
       COUNT(Registration.Registration_ID) AS Total_Registrations  
FROM `Event`  
LEFT JOIN Registration  
      ON `Event`.Event_ID = Registration.Event_ID  
GROUP BY `Event`.Event_ID, `Event`.Event_Name;  
  
SELECT `Event`.Event_Name,  
       COUNT(Registration.Registration_ID) AS Total_Students  
FROM `Event`  
INNER JOIN Registration  
      ON `Event`.Event_ID = Registration.Event_ID  
GROUP BY `Event`.Event_ID, `Event`.Event_Name  
HAVING COUNT(Registration.Registration_ID) > 1;
```

15. Subquery

```
SELECT Student_Name  
FROM Student  
WHERE Student_ID IN  
(  
  SELECT Student_ID  
  FROM Registration  
  WHERE Event_ID =  
    (  
      SELECT Event_ID  
      FROM `Event`  
      WHERE Event_Name = 'Coding Competition'  
    )  
);
```

16. View

```
CREATE VIEW Student_Event_View AS  
SELECT Student.Student_Name, Student.Branch,  
       `Event`.Event_Name, `Event`.Event_Date, `Event`.Venue  
FROM Registration  
INNER JOIN Student  
      ON Registration.Student_ID = Student.Student_ID  
INNER JOIN `Event`  
      ON Registration.Event_ID = `Event`.Event_ID;  
  
SELECT * FROM Student_Event_View;
```

17. Index

```
CREATE INDEX idx_event_name  
ON `Event`(Event_Name);  
  
SHOW INDEX FROM `Event`;
```

18. Trigger

```
DELIMITER //  
  
CREATE TRIGGER after_registration  
AFTER INSERT ON Registration  
FOR EACH ROW  
BEGIN  
  INSERT INTO Attendance  
    (Attendance_ID, Registration_ID, Status)  
  VALUES  
    (NEW.Registration_ID + 100,  
     NEW.Registration_ID,  
     'Pending');  
END //  
  
DELIMITER ;  
  
INSERT INTO Registration  
(Registration_ID, Student_ID, Event_ID, Registration_Date)  
VALUES  
(306, 101, 204, '2026-09-25');  
  
SELECT * FROM Attendance;
```

19. CRUD Operations

```
-- CREATE
INSERT INTO Student
(Student_ID, Student_Name, Branch, `Year`, Email)
VALUES
(106, 'Karan Mehta', 'IT', 2, 'karan@gmail.com');

-- READ
SELECT * FROM Student;

-- UPDATE
UPDATE Student
SET Branch = 'CSE'
WHERE Student_ID = 106;

SELECT * FROM Student WHERE Student_ID = 106;

-- DELETE
DELETE FROM Student
WHERE Student_ID = 106;

SELECT * FROM Student;
```